Assignment 4: Load Balancing & Scaling on Local Machine

Overview

This assignment provides a hands-on experience in load balancing and scaling through local windows / Linux /macOS machine.

Deliverables

Each student must submit the following:

Presentation Slides

- Title slide with student name(s), course, and assignment title.
- Clear explanation of each completed task:
- Screenshots of key steps / results.
- Lessons learned and challenges encountered.

Presentation Guidelines

- Use professional formatting (legible fonts, consistent colors, organized layout).
- Ensure clarity: avoid overcrowding slides with text; use bullet points and visuals.
- Provide screenshots as evidence of work completed.

1. Task Completion (70 points total)

Step 1: Go to Python's official website

- Download Python 3.x https://www.python.org/downloads/windows/
- IMPORTANT During installation:
  - Check "Add Python to PATH"

Step 2: Verify installation

- Open Command Prompt (cmd) **or** PowerShell
  - Run the command: python –version
  - Run the command: pip --version
  - Install pip if missing

Step 3: Create project folder

- Open command prompt and run this command to create folder
  - mkdir load-balancing-lab
- Navigate to the folder which you have created using the command below:
  - cd load-balancing-lab

Step 4: Create server file

- Open Notepad (or VS Code if installed)
- Save as: server.py

```
from http.server import BaseHTTPRequestHandler, HTTPServer
import sys
PORT = int(sys.argv[1])
INSTANCE = sys.argv[2]

class Handler(BaseHTTPRequestHandler):
  def do_GET(self):
    self.send_response(200)
    self.end_headers()
    message = f"Hello from instance {INSTANCE}"
    self.wfile.write(message.encode())

server = HTTPServer(("localhost", PORT), Handler)
print(f"Running instance {INSTANCE} on port {PORT}")
server.serve_forever()
```

Step 5: Run multiple servers, and simulate scaling.

- Open 3 Command Prompt individually in your windows machine from different port
  - python server.py 8001 1
  - python server.py 8002 2
  - python server.py 8003 3

Step 6: Open browser and Test in browser

- http://localhost:8001
- http://localhost:8002
- http://localhost:8003

Each shows different instance

Step 7: open another command prompt and Install requests library

- pip install requests

Step 8: Create load balancer

- Create a load_balancer.py file.

```
import itertools
from http.server import BaseHTTPRequestHandler, HTTPServer
import requests

servers = [
   "http://localhost:8001",
   "http://localhost:8002",
   "http://localhost:8003"
]

cycle = itertools.cycle(servers)

class Handler(BaseHTTPRequestHandler):
   def do_GET(self):
      target = next(cycle)
      response = requests.get(target)

      self.send_response(200)
      self.end_headers()
      self.wfile.write(response.content)

server = HTTPServer(("localhost", 9000), Handler)
print("Load balancer running on port 9000")
server.serve_forever()
```

Step 9: Open another command prompt and Run load balancer

- python load_balancer.py

Step 10: Demonstrate load balancing

- Open browser and test the following URL http://localhost:9000
- Refresh multiple times. You'll see:
    - Hello from instance 1
    - Hello from instance 2
    - Hello from instance 3
- This proves load balancing works on local machine. This is round-robin load balancing. Requests are distributed evenly

Step 11: Demonstrate Scaling

- More servers = more capacity. So add one more server.
- Open new command prompt and run *python server.py 8004 4*
- Update the file load_balancer.py by:

- servers = [
  "http://localhost:8001",
  "http://localhost:8002",
  "http://localhost:8003",
  "http://localhost:8004"
  ]

- Now restart the load balancer again by *python load_balancer.py*
- Now 4 servers handle traffic

Step 12: Demonstrate Slow Server Problem.

- Modify server.py. by
  - import time
    time.sleep(3)
- Restart instance 2 (Test only for instance 2). Sometimes response is slow. Explain uneven performance

Step 13: Demonstrate Failure Handling

- Stop any one server by
  - Ctrl + C
- Now refresh browser. You will see it may crash or hang.
- Explain real systems must handle failures

Step 14: Improve Load Balancer (basic fault tolerance)

- By taking this step dead servers are skipped.
- Update the load balancer by modifying do_GET method in load_balancer.py

```
class Handler(BaseHTTPRequestHandler):
  def do_GET(self):
    for _ in range(len(servers)):
      target = next(cycle)
      try:
        response = requests.get(target, timeout=1)
        self.send_response(200)
        self.end_headers()
        self.wfile.write(response.content)
        return
      except:
        continue
    # If all servers fail
    self.send_response(500)
    self.end_headers()
    self.wfile.write(b"All servers are down")
```

- Stop and restart python load_balancer.py

- You will observe if one server is down → system still works
- If all servers are down → error message shown

This is a real-world concept used in systems such as AWS load balancer, NGINX etc webserver.

2. Presentation Quality (20 points total)

- Clear structure (intro, results, discussion, conclusion)
- Slides visually organized and professional, for example, adding pictures, transitions, animations
- Screenshots or videos included as evidence

3. Professionalism & Delivery (10 points total)

- Confident and clear presentation deliver
- **Submission Instructions**
  - Submit slides (PowerPoint or PDF) to Moodle by the deadline.